CSS 262: Linux Administration

Containerization (Docker)

Lecture 12: Containers, Images & Security

📋 Today's Agenda

Part 1: Container Fundamentals

- VMs vs Containers
- Linux namespaces & cgroups
- How Docker uses the kernel

Part 2: Docker Images & Registry

- Images, layers, and Dockerfile
- Building and tagging images
- Docker Hub and private registries

Part 3: Running & Managing Containers

- `docker run`, lifecycle commands
- Volumes, networking, port mapping
- Docker Compose

Part 4: Container Security

- Attack surface: images, runtime, daemon
- Rootless Docker, read-only containers
- Scanning images for CVEs

🔄 Quick Recap: Week 11

Logging & Auditing

- `journalctl` : indexed binary log, filter by unit/priority
- `rsyslog` : facility.severity routing, remote forwarding
- `logrotate` : prevent disk fill with rotation policy
- `auditd` : kernel-driven syscall & file watches

Key Takeaways

- Logs are your evidence after an incident
- `ausearch` + `aureport` for querying audit events
- Cron syntax: 5 time fields + command
- Audit all crontabs during incident response

🐳 **Today:** Containers package applications with their dependencies – consistent, portable, and fast. But they introduce new security considerations.

Part 1

🐳 Container Fundamentals

🐳 VMs vs Containers

Virtual Machines

- Full OS kernel per VM (GBs of disk)
- Hardware emulation via hypervisor
- Boot time: minutes
- Strong isolation (separate kernel)
- Heavy: memory and CPU overhead

Containers

- **Share the host kernel** – no emulation
- Isolated processes with namespaces & cgroups
- Start in milliseconds
- Lighter isolation (same kernel)
- Lightweight: MBs, much lower overhead

⚠️ **Security implication:** A kernel vulnerability can affect all containers on the host. VMs provide stronger isolation boundaries because each has its own kernel.

🔧 Linux Namespaces — Process Isolation

Namespaces are the kernel feature that makes containers possible. Each container gets its own view of the system.

Namespace	Flag	Isolates
`pid`	`CLONE_NEWPID`	Process IDs — container sees PID 1 as its init
`net`	`CLONE_NEWNET`	Network interfaces, routing tables, ports
`mnt`	`CLONE_NEWNS`	Filesystem mount points
`uts`	`CLONE_NEWUTS`	Hostname and domain name
`ipc`	`CLONE_NEWIPC`	System V IPC, POSIX message queues
`user`	`CLONE_NEWUSER`	UID/GID mappings (rootless containers)
`cgroup`	`CLONE_NEWCGROUP`	cgroup root view

💡 Run `lsns` on the host to list all active namespaces. Each container process has its own namespace set.

🔧 cgroups — Resource Limits

What cgroups Do

- **Control Groups** — kernel feature for resource management
- Limit CPU, memory, I/O, network bandwidth per group
- Docker maps each container to a cgroup
- Prevents a container from consuming all host resources
- cgroups v2 is default on modern systems

Docker Resource Flags

```
# Limit to 512 MB RAM
docker run -m 512m nginx

# Limit to 1 CPU
docker run --cpus="1.0" nginx

# Limit memory + disable swap
docker run -m 256m --memory-swap 256m nginx

# View container cgroup
cat /sys/fs/cgroup/memory/docker/
```

🚨 **Always set resource limits in production.** Without limits, a container (or a compromised container) can exhaust host memory and crash all services.

Part 2

📦 Docker Images & Registry

📦 Docker Images

What Is an Image?

- Read-only template with the app + dependencies
- Built from a **Dockerfile** – layered instructions
- Each instruction adds a new layer (union filesystem)
- Layers are cached and shared between images
- Image = snapshot; Container = running instance

Image Naming

```
registry/repository:tag

docker.io/library/nginx:latest
ghcr.io/myorg/myapp:v1.2.3
192.168.1.100:5000/myapp:dev
```

💡 `latest` is just a tag – it does NOT guarantee the most recent version. Always pin to a specific version tag in production (e.g., `nginx:1.25.3`).

📝 Dockerfile — Building Images

```dockerfile
# Base image — always pin a specific tag
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Copy dependency list first (cache optimization)
COPY requirements.txt .

# Install dependencies (this layer is cached if requirements.txt is unchanged)
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Run as non-root user (security!)
RUN useradd -r -u 1001 appuser
USER appuser

# Document the port (informational only — does not publish)
EXPOSE 8000

# Default command
CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

🔨 Building & Managing Images

```
# Build image from Dockerfile in current directory
docker build -t myapp:v1.0 .

# Build with a different Dockerfile
docker build -f Dockerfile.prod -t myapp:prod .

# List local images
docker images
docker image ls

# Remove an image
docker image rm myapp:v1.0
docker rmi myapp:v1.0

# Pull from registry
docker pull nginx:1.25
docker pull python:3.11-slim

# Push to registry
docker login
docker push myorg/myapp:v1.0

# Inspect image layers
```

🏗️ Dockerfile Best Practices

Layer Caching — Order Matters

```dockerfile
# ✅ Good: copy deps first, code second
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .

# ❌ Bad: changes to any file bust the pip cache
COPY . .
RUN pip install -r requirements.txt
```

Multi-Stage Builds — Smaller Final Images

```dockerfile
# Build stage (has compiler, dev tools)
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o server .

# Final stage (minimal runtime, no compiler)
FROM gcr.io/distroless/base
COPY --from=builder /app/server /server
ENTRYPOINT ["/server"]
```

Part 3

🚀 Running & Managing Containers

🚀 `docker run` – Core Options

```
# Basic run (foreground)
docker run nginx

# Detached (background)
docker run -d nginx

# Named container
docker run -d --name webserver nginx

# Port mapping: host:container
docker run -d -p 8080:80 nginx

# Environment variables
docker run -d -e MYSQL_ROOT_PASSWORD=secret mysql:8

# Interactive shell
docker run -it ubuntu bash

# Remove container when it exits
docker run --rm ubuntu echo "hello"

# Mount a host directory (bind mount)
docker run -d -v /host/data:/app/data nginx

```

🔄 Container Lifecycle

```
# List running containers
docker ps

# List all containers (including stopped)
docker ps -a

# Stop a container gracefully (SIGTERM → SIGKILL after timeout)
docker stop webserver

# Start a stopped container
docker start webserver

# Restart
docker restart webserver

# Remove a stopped container
docker rm webserver

# Remove running container forcefully
docker rm -f webserver

# View logs
docker logs webserver
docker logs -f webserver        # Follow
```

💾 Volumes & Bind Mounts

Volumes (Recommended)

```
# Create a named volume
docker volume create mydata

# Use in container
docker run -d \
  -v mydata:/var/lib/mysql \
  mysql:8

# List volumes
docker volume ls

# Inspect volume (find mount path)
docker volume inspect mydata

# Remove unused volumes
docker volume prune
```

Bind Mounts

```
# Mount host directory
docker run -d \
  -v /host/config:/etc/nginx/conf.d:ro \
  nginx

# Read-only flag prevents container
# from modifying host files
```

Key Differences

	Volume	Bind Mount
Managed by	Docker	OS
Portability	✅ High	❌ Host-specific
Backup	`docker cp`	Direct

🌐 Docker Networking

Default Networks

```
docker network ls
# NETWORK ID   NAME      DRIVER    SCOPE
# abc123       bridge    bridge    local   ← default
# def456       host      host      local
# ghi789       none      null      local
```

Creating Custom Networks

```
# Create a network
docker network create mynet

# Connect containers — they resolve each other by name!
docker run -d --name db --network mynet postgres:15
docker run -d --name app --network mynet -e DB_HOST=db myapp

# Inspect network
docker network inspect mynet
```

💡 Containers on the same user-defined network can reach each other by container name. This is DNS-based service discovery built into Docker.

🐙 Docker Compose

Define and run multi-container applications with a single YAML file.

```yaml
# docker-compose.yml
version: "3.9"

services:
  db:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      POSTGRES_PASSWORD: secret
    restart: unless-stopped

  app:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
    environment:
      DATABASE_URL: postgresql://postgres:secret@db/appdb
    restart: unless-stopped
```

Part 4

🔒 Container Security

🔒 Container Threat Model

Attack Surface

- **Image vulnerabilities** – outdated base image packages (CVEs)
- **Privileged containers** – `--privileged` breaks isolation
- **Running as root** – UID 0 inside = UID 0 on host (without user ns)
- **Docker daemon** – root-owned UNIX socket, accessible = root
- **Exposed secrets** – env vars, Dockerfile RUN commands in history
- **Unprotected registries** – pulling malicious images

Defense Layers

- Pin and scan base images
- Non-root USER in Dockerfile
- Read-only filesystem (`--read-only`)
- Drop Linux capabilities (`--cap-drop`)
- Seccomp profiles to limit syscalls
- Rootless Docker (daemon runs as user)
- Network segmentation (custom networks)

🚨 The Docker daemon socket `/var/run/docker.sock` is effectively a root backdoor. Never mount it inside a container unless absolutely necessary.

🔒 Running Containers Securely

```
# Drop ALL capabilities, add back only what you need
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE nginx

# Read-only root filesystem (mount /tmp separately)
docker run --read-only --tmpfs /tmp nginx

# Run as a specific non-root user (UID 1001)
docker run --user 1001:1001 myapp

# No new privileges (prevents privilege escalation via SUID)
docker run --security-opt no-new-privileges myapp

# Apply a seccomp profile (restrict allowed syscalls)
docker run --security-opt seccomp=/etc/docker/seccomp.json myapp

# Combine for a hardened container
docker run -d \
  --user 1001:1001 \
  --read-only \
  --tmpfs /tmp \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  -m 256m --cpus="0.5" \
  myapp
```

🔍 Scanning Images for CVEs

Trivy – Fast, Free Image Scanner

```
# Install Trivy
sudo apt install trivy   # or: curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install

# Scan an image
trivy image nginx:1.25
trivy image python:3.11-slim

# Scan local Dockerfile
trivy config Dockerfile

# Output only HIGH and CRITICAL CVEs
trivy image --severity HIGH,CRITICAL nginx:latest

# JSON output for automation
trivy image -f json -o results.json myapp:v1.0
```

Built-in: Docker Scout

```
docker scout cves nginx:latest
docker scout recommendations nginx:latest   # Suggests safer base images
```

🔍 Inspecting Running Containers

```
# Show container resource usage live
docker stats

# Show container processes (from host perspective)
docker top webserver

# View full container config (mounts, env, network)
docker inspect webserver

# Check what files changed from image baseline
docker diff webserver

# Copy files out of a container
docker cp webserver:/etc/nginx/nginx.conf ./nginx.conf

# View image layer history (check for secrets in RUN commands!)
docker history myapp:v1.0 --no-trunc
```

💡 `docker history --no-trunc` reveals every command that was run during the build – including any passwords or tokens accidentally baked in.

🔒 Rootless Docker

Why Rootless?

- The traditional Docker daemon runs as `root` – a compromised container can become root on the host
- Rootless mode maps container root to an unprivileged user on the host via user namespaces

```
# Install rootless Docker
dockerd-rootless-setuptool.sh install

# Use it — same commands, different socket
export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/docker.sock
docker run -d nginx

# Enable user systemd service
systemctl --user enable --now docker
```

Checking Who Owns the Socket

```
ls -la /var/run/docker.sock
# srw-rw——— 1 root docker 0 … /var/run/docker.sock
# Being in the 'docker' group = effective root access
```

🚨 Adding a user to the `docker` group grants them root-equivalent access. Treat docker group membership like sudoers.

🔒 Secrets Management in Docker

❌ Wrong Ways

```
# BAD: Secret visible in docker history
RUN curl -H "Authorization: Bearer $TOKEN" https://api.example.com

# BAD: Hardcoded in image
ENV DB_PASSWORD=mysecretpassword
```

✅ Right Ways

```
# 1. Docker secrets (Swarm mode)
echo "mysecret" | docker secret create db_password -
docker service create --secret db_password myapp

# 2. Environment variables at runtime (not in image)
docker run -e DB_PASSWORD="$DB_PASSWORD" myapp

# 3. Volume-mounted secrets files
docker run -v /run/secrets:/run/secrets:ro myapp

# 4. Use a secrets manager (Vault, AWS Secrets Manager)
```

📊 Container Logging & Monitoring

```
# Container stdout/stderr → Docker log driver
# Default: json-file driver
docker logs myapp

# With journald driver, logs go to systemd journal
# /etc/docker/daemon.json:
{
  "log-driver": "journald"
}

# Then access via journalctl
journalctl -u docker CONTAINER_NAME=myapp

# Inspect container metrics (one-shot)
docker stats --no-stream

# Format output
docker stats --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```

💡 Centralize container logs to a remote SIEM or ELK stack. The journald log driver integrates nicely with the rsyslog forwarding you configured in Week 11.

✅ Docker Security Checklist

Image Hygiene

- ☐ Pin base image to specific digest or tag
- ☐ Use minimal base images (alpine, distroless)
- ☐ Multi-stage builds to remove build tools
- ☐ Scan with Trivy before pushing
- ☐ No secrets in Dockerfile or image layers

Runtime Hardening

- ☐ `USER` set to non-root in Dockerfile
- ☐ `--cap-drop ALL` + add back only needed
- ☐ `--read-only` filesystem where possible
- ☐ `--security-opt no-new-privileges`
- ☐ Resource limits: `-m` and `--cpus`
- ☐ Never mount `/var/run/docker.sock`
- ☐ Custom networks (not default bridge)

📊 Summary: Docker & Container Security

Key Concepts

1. **Namespaces**: PID, net, mnt, user isolation
2. **cgroups**: CPU, memory, I/O limits
3. **Image**: layered read-only template (Dockerfile)
4. **Container**: running image instance
5. **Volume**: persistent storage beyond container lifetime
6. **Docker Compose**: multi-container app definition

Security Principles

Risk	Mitigation
Vulnerable packages	Scan with Trivy
Root inside container	`USER` directive
Excess capabilities	`--cap-drop ALL`
Writable filesystem	`--read-only`
Uncontrolled resources	`-m` / `--cpus`
Secrets in images	Runtime env / secrets mgr

🎯 Learning Objectives: Did We Achieve?

By now, you should be able to:

- ✅ Explain how namespaces and cgroups enable container isolation
- ✅ Distinguish containers from VMs in terms of isolation and performance
- ✅ Write a Dockerfile with security best practices (non-root user, multi-stage)
- ✅ Build, tag, and push Docker images
- ✅ Run containers with resource limits and security flags
- ✅ Use volumes for persistent storage
- ✅ Define multi-container apps with Docker Compose
- ✅ Scan images for CVEs using Trivy
- ✅ Explain the Docker daemon socket security risk
- ✅ Avoid embedding secrets in images

🎓 **Next Week:** Vulnerability Scanning & Patching – systematic approaches to finding and fixing vulnerabilities across your Linux fleet.

🧪 Lab Practice: Docker Hardening

Exercise 1: Build & Run a Container

Write a Dockerfile for a Python Flask app. Build the image, run it, and map port 5000. Verify it responds with `curl localhost:5000`.

Exercise 2: Security Hardening

Take the container from Ex 1 and run it with: non-root user, read-only filesystem, `--cap-drop ALL`, and a 256 MB memory limit. Verify it still functions.

Exercise 3: Image Scanning

Pull `python:3.10` and scan it with Trivy. Count HIGH and CRITICAL CVEs. Compare with `python:3.11-slim`. Which has fewer vulnerabilities? Why?

Exercise 4: Docker Compose

Write a `docker-compose.yml` that runs a web app and a PostgreSQL database on a custom network. The DB should use a named volume. Test with `docker compose up -d`.

🔗 Additional Resources

Documentation

- Docker Documentation – Official reference
- Dockerfile Best Practices – Official guide
- Trivy – Image & config vulnerability scanner
- Docker Bench for Security – CIS Docker Benchmark script

Books & Guides

- *"Container Security"* by Liz Rice – Comprehensive security-focused coverage
- CIS Docker Benchmark – Hardening checklist
- NIST SP 800-190 – Application Container Security Guide

Practice

- Run Docker Bench for Security against your Docker installation
- Try escaping a privileged container (in a lab VM) to understand the risk
- Build a distroless image for a Go application

Questions?

Next: Vulnerability Scanning & Patching